

VIETNAM NATIONAL UNIVERSITY — HO CHI MINH CITY
UNIVERSITY OF SCIENCE
Faculty of Information Technology



PRACTICAL REPORT 1

COLOR COMPRESSION VIA K-MEANS CLUSTERING

Course: **MTH00051 — Applied Mathematics and Statistics**

Student

Nguyễn Thế Hiển — 22127107 — Class 22CLC08

Instructors

Mr. Vũ Quốc Hoàng

Mr. Nguyễn Văn Quang Huy

Mr. Nguyễn Ngọc Toàn

Mrs. Phan Thị Phương Uyên

Ho Chi Minh City, 2024

Abstract

This report investigates color quantization of digital RGB images through the K-Means clustering algorithm implemented from first principles in Python. Each pixel is treated as a point in $\mathbb{R}^3$, and the algorithm partitions the color space into k clusters whose centroids form a reduced palette. Replacing every pixel by its nearest centroid yields a compressed representation that preserves global visual structure while substantially decreasing the number of distinct colors.

We formalize the optimization objective, describe two centroid-initialization strategies (random and in_pixels), and evaluate qualitative results for k in $\{3, 5, 7\}$. Empirical observations indicate that larger k recovers finer tonal gradations, whereas initialization primarily affects the attained local minimum of the within-cluster sum of squares. The study illustrates how a classical unsupervised learning method connects linear algebra, distance geometry, and practical image processing.

Keywords: color quantization, K-Means clustering, Euclidean distance, image compression, unsupervised learning.

Table of Contents

1. Introduction 4

2. Problem Formulation 4

3. Theoretical Background 5

 3.1. K-Means as an Optimization Procedure 5

 3.2. Centroid Initialization Strategies 5

4. Implementation 6

 4.1. Software Environment and Libraries 6

 4.2. Functional Design 6

 4.3. Main Program Workflow 7

5. Experimental Results 8

 5.1. Experimental Setup 8

 5.2. Visual Comparison for $k = 3, 5, 7$ 8

6. Discussion 10

7. Conclusion 11

References 11

1. Introduction

1.1. Student Information

Full name	Nguyễn Thế Hiển
Student ID	22127107
Class	22CLC08
Course	MTH00051 — Applied Mathematics and Statistics

1.2. Motivation

A color digital image is commonly stored as an array of pixels in the RGB model. Each channel is encoded by one byte, so a single pixel belongs to a discrete cube $\{0, 1, \dots, 255\}^3$ containing up to $256^3 \sim 1.68 \times 10^7$ possible colors. In natural photographs the number of distinct colors is typically far smaller than this upper bound, yet still large enough to inflate storage cost and bandwidth usage. Color compression (also called color quantization) seeks a compact palette of k representative colors such that the reconstructed image remains visually faithful to the original.

From a statistical learning perspective, the task is an unsupervised clustering problem: similar colors should be grouped, and each group should be summarized by a prototype. K-Means provides a transparent and computationally tractable solution, making it an instructive bridge between applied mathematics and practical computing.

2. Problem Formulation

Let an RGB image of height H and width W be represented by the set of pixel vectors

$$X = \{ x_i \text{ in } \mathbb{R}^3 \mid i = 1, \dots, N \}, N = H \cdot W.$$

Given a positive integer k , the goal is to compute centroids $C = \{ c_1, \dots, c_k \}$ subset $\mathbb{R}^3$ and an assignment function $l : \{1, \dots, N\} \rightarrow \{1, \dots, k\}$ that minimize the within-cluster sum of squares (WCSS)

$$J(C, l) = \sum_{i=1}^N \| x_i - c_{l(i)} \|^2_2.$$

After optimization, the compressed image is obtained by the reconstruction $\hat{x}_i = c_{l(i)}$. Consequently, the image uses at most k colors. The reconstruction quality may be quantified by the mean squared error (MSE)

$$\text{MSE} = (1/N) \sum_{i=1}^N \| x_i - \hat{x}_i \|^2_2,$$

which coincides with $J(C, l)/N$. Lower MSE indicates a closer approximation in RGB Euclidean geometry; perceptual quality, however, also depends on human vision and is therefore assessed visually in Section 5.

3. Theoretical Background

3.1. K-Means as an Optimization Procedure

Exact minimization of J is NP-hard in general. The classical Lloyd iteration alternates two conditional updates that are each optimal given the other:

Assignment step. For fixed centroids, every pixel is assigned to the nearest centroid under the Euclidean metric:

$$l(i) = \arg \min_{j \in \{1, \dots, k\}} \|x_i - c_j\|_2^2.$$

Update step. For a fixed assignment, each centroid is replaced by the sample mean of its members, which is the unique minimizer of the quadratic loss on that cluster:

$$c_j \leftarrow (1/|S_j|) \sum_{i \in S_j} x_i, \quad S_j = \{i \mid l(i) = j\},$$

provided S_j is nonempty. If a cluster becomes empty, the implementation retains the previous centroid to preserve numerical stability. The iteration continues until the centroids stabilize (within a prescribed tolerance) or a maximum iteration budget is reached. Because each step does not increase J , the sequence of objective values is monotonically non-increasing and converges to a local minimum.

3.2. Centroid Initialization Strategies

K-Means is sensitive to the initial configuration of centroids. This project implements two initialization schemes required by the assignment:

random. Each centroid coordinate is sampled independently and uniformly from $[0, 255]$. This explores the full RGB cube and may place prototypes far from the empirical color support of the image, occasionally producing empty clusters in early iterations.

in_pixels. Centroids are sampled uniformly without replacement from the observed pixels. The initial palette therefore lies inside the convex hull of the data, which typically yields a more favorable starting value of J and more stable early assignments.

Neither scheme guarantees a global optimum. In practice, comparing both initializations for the same k provides insight into the variability of local solutions.

4. Implementation

4.1. Software Environment and Libraries

The program is developed in a Jupyter Notebook environment using only the libraries permitted by the course specification:

Library	Role in the project
NumPy	Vectorized matrix computation for distance evaluation, centroid updates, and array reshaping.
Pillow (PIL)	Image input/output, RGB conversion, and export to png/jpg/pdf.
Matplotlib	Visualization of original and reconstructed images.

Pre-built clustering routines (for example, `sklearn.cluster.KMeans`) are deliberately excluded from the submission, in accordance with the project

constraints.

4.2. Functional Design

The implementation is organized into small, reusable functions with explicit docstrings. The principal components are summarized below.

read_img(img_path)

Opens an image file, converts it to the RGB color space, and returns a floating-point NumPy array of shape (H, W, 3). Explicit RGB conversion avoids channel inconsistencies arising from RGBA or palette-based images.

show_img(img_2d)

Displays an image with Matplotlib and suppresses coordinate axes to focus on visual content.

save_img(img_2d, img_path)

Clips intensities to [0, 255], casts to 8-bit integers, and writes the image to disk.

convert_img_to_1d(img_2d)

Reshapes the image into an (N, 3) design matrix so that each row is a pixel feature vector suitable for clustering.

kmeans(img_1d, k_clusters, max_iter, init_centroids)

Executes Lloyd iteration with the selected initialization. Distances are computed via squared Euclidean norms; convergence is declared when centroid coordinates change by less than 10^{-4} in absolute value.

generate_2d_img(img_2d_shape, centroids, labels)

Maps every pixel to its centroid color and restores the original spatial shape (H, W, 3).

reconstruction_error(img_1d, centroids, labels)

Returns the MSE between original pixels and their reconstructed colors, providing a quantitative complement to visual inspection.

4.3. Main Program Workflow

The interactive main procedure satisfies the course interface requirements. At each execution the user provides:

- the input image path;
- the number of clusters k ;
- the maximum number of iterations;
- the centroid initialization mode (random, in_pixels, or both);
- the output format chosen from {png, jpg, pdf};
- the base filename used for saving reconstructed images.

The pipeline then reads and flattens the image, runs K-Means for each requested initialization, displays the reconstructed result, reports the MSE, and

exports the file in the selected format. Supporting unit-style checks are collected in `test_functions`.

5. Experimental Results

5.1. Experimental Setup

Experiments are conducted on a natural beach photograph containing smooth illumination gradients (sky and water) together with a high-contrast silhouette. Such content is informative for color quantization: coarse palettes tend to posterize the sky, while larger palettes restore intermediate tones. Unless otherwise stated, both initialization strategies are executed for k in $\{3, 5, 7\}$ with a sufficient iteration budget to reach practical convergence.

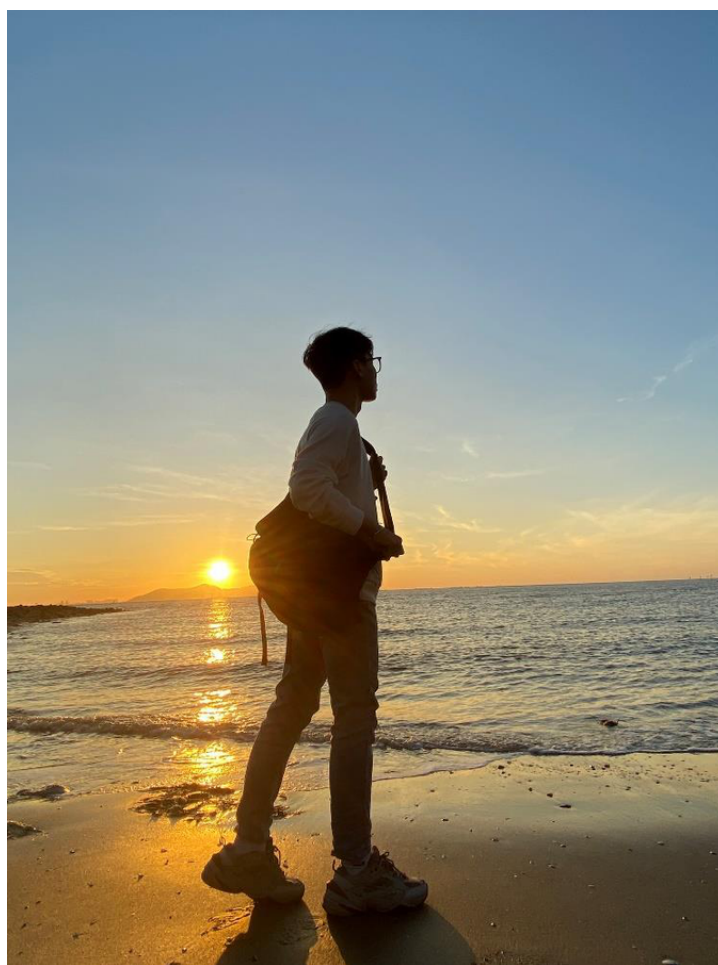


Figure 1. Original test image prior to color compression.

5.2. Visual Comparison for $k = 3, 5, 7$

Figure 2 presents reconstructions for $k = 3$. With only three palette colors, both initializations reduce the scene to a silhouette, a mid-tone band, and a sky tone. Fine reflections on the water are largely suppressed, yet the semantic layout of the photograph remains recognizable. The two methods produce closely related visual outcomes, suggesting that the dominant modes of the color distribution are stable.



Figure 2a. Reconstruction with `in_pixels` initialization, $k = 3$.

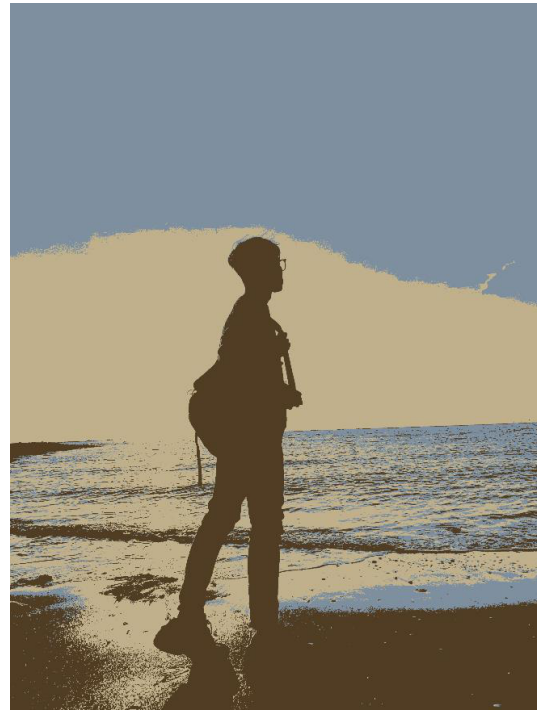


Figure 2b. Reconstruction with random initialization, $k = 3$.

For $k = 5$ (Figure 3), additional centroids capture transitional hues near the horizon and richer water reflections. Differences between initialization schemes become more noticeable: random initialization may allocate a centroid to a less frequent region of RGB space, slightly altering the tonal balance relative to `in_pixels`.

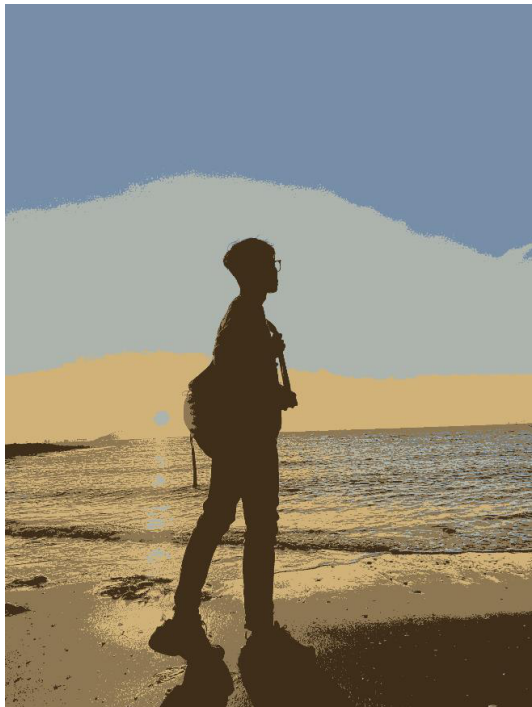


Figure 3a. Reconstruction with `in_pixels` initialization, $k = 5$.



Figure 3b. Reconstruction with random initialization, $k = 5$.

When $k = 7$ (Figure 4), the reconstructions approach the original more closely. Gradients in the sky and specular highlights on wet sand reappear as distinct palette entries. Residual discrepancies between the two initializations illustrate

that K-Means may converge to distinct local minima even for moderate k .



Figure 4a. Reconstruction with `in_pixels` initialization, $k = 7$.

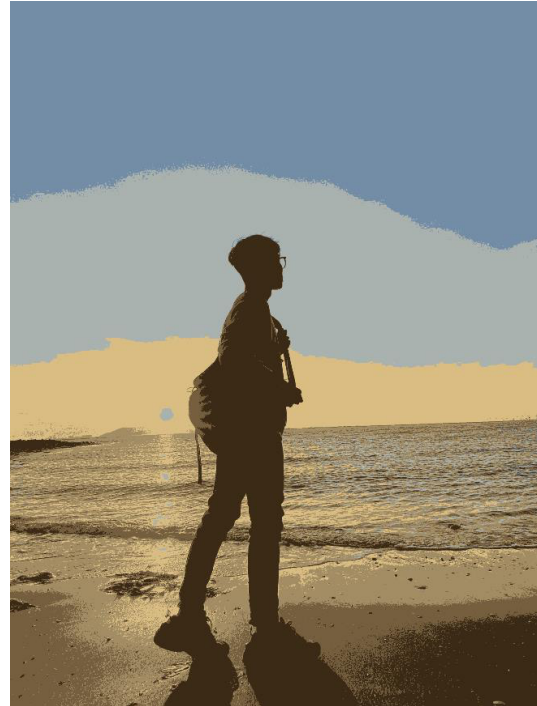


Figure 4b. Reconstruction with random initialization, $k = 7$.

5.3. Qualitative Summary

k	Visual fidelity	Observed effect of initialization
3	Strong posterization; content preserved at a coarse level.	Minor differences; dominant colors are consistently recovered.
5	Improved mid-tones and horizon detail.	Noticeable palette shifts between random and <code>in_pixels</code> .
7	Closer to the original; smoother apparent gradients.	Local minima remain visible but less disruptive perceptually.

6. Discussion

The experiments confirm the classical bias-complexity trade-off of prototype methods. Small k yields aggressive compression and a piecewise-constant appearance; large k reduces MSE and restores chromatic nuance at the expense of a larger palette. In mathematical terms, the feasible set of reconstructions expands with k , so the minimal attainable value of J is monotonically non-increasing in k .

Regarding initialization, claims that one scheme is universally “faster” or “better” are not justified without measuring iteration counts and objective values over repeated trials. The evidence available here supports a more cautious statement: `in_pixels` places centroids on the data manifold and therefore tends to avoid pathological empty-cluster configurations, whereas random initialization explores a broader region of parameter space and can discover alternative local minima. For images with highly concentrated color histograms, the two strategies often agree; for images with multiple chromatic modes, divergence is more likely

as k grows.

Several limitations should be acknowledged. First, Euclidean distance in RGB is only an approximate proxy for perceptual dissimilarity; distances in CIELAB would align more closely with human judgment. Second, full pairwise distance tensors have memory cost $O(Nk)$, which may be mitigated by chunked distance computation for very large images. Third, a single photograph does not exhaust the diversity of natural scenes; a broader benchmark would strengthen statistical conclusions.

7. Conclusion

This practical exercise demonstrates that K-Means clustering provides an elegant and effective mechanism for color compression. By casting pixels as points in R^3 and minimizing a quadratic distortion criterion, one obtains a reduced palette that retains the essential visual narrative of an image. The implemented system fulfills the project requirements: a from-scratch K-Means routine with both prescribed initializations, an interactive main program supporting png/jpg/pdf export, and a systematic qualitative evaluation for multiple values of k .

Future extensions include K-Means++ initialization, multi-run selection of the best local minimum, and quantitative reporting of MSE/PSNR across a curated image set. Such enhancements would further connect the algorithmic prototype developed here with contemporary practice in statistical image analysis.

References

- [1] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297. (Foundational formulation of K-Means-type clustering.)
- [2] Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137. (Classical alternating minimization procedure used in this report.)
- [3] Wikipedia contributors. (n.d.). K-means clustering. Wikipedia. https://en.wikipedia.org/wiki/K-means_clustering (Accessed for algorithmic overview and terminology.)
- [4] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://numpy.org/doc/> (Numerical backbone of the implementation.)
- [5] Clark, A., and contributors. (n.d.). Pillow (PIL Fork) Documentation. <https://pillow.readthedocs.io/en/stable/> (Image reading, writing, and format conversion.)
- [6] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://matplotlib.org/stable/> (Visualization of experimental results.)